

Sesión 14: Modelado e Integración

Integración de Capas en MVC

IPC 2 | Unidad 3 | Ingeniería USAC

El Flujo de Información



Conectando la lógica, los datos y el usuario.

Objetivos de la Sesión



Arquitectura

Implementar el desacoplamiento lógico entre las capas de la aplicación web.



Integración

Conectar dinámicamente los Modelos con los Controladores y las Vistas.

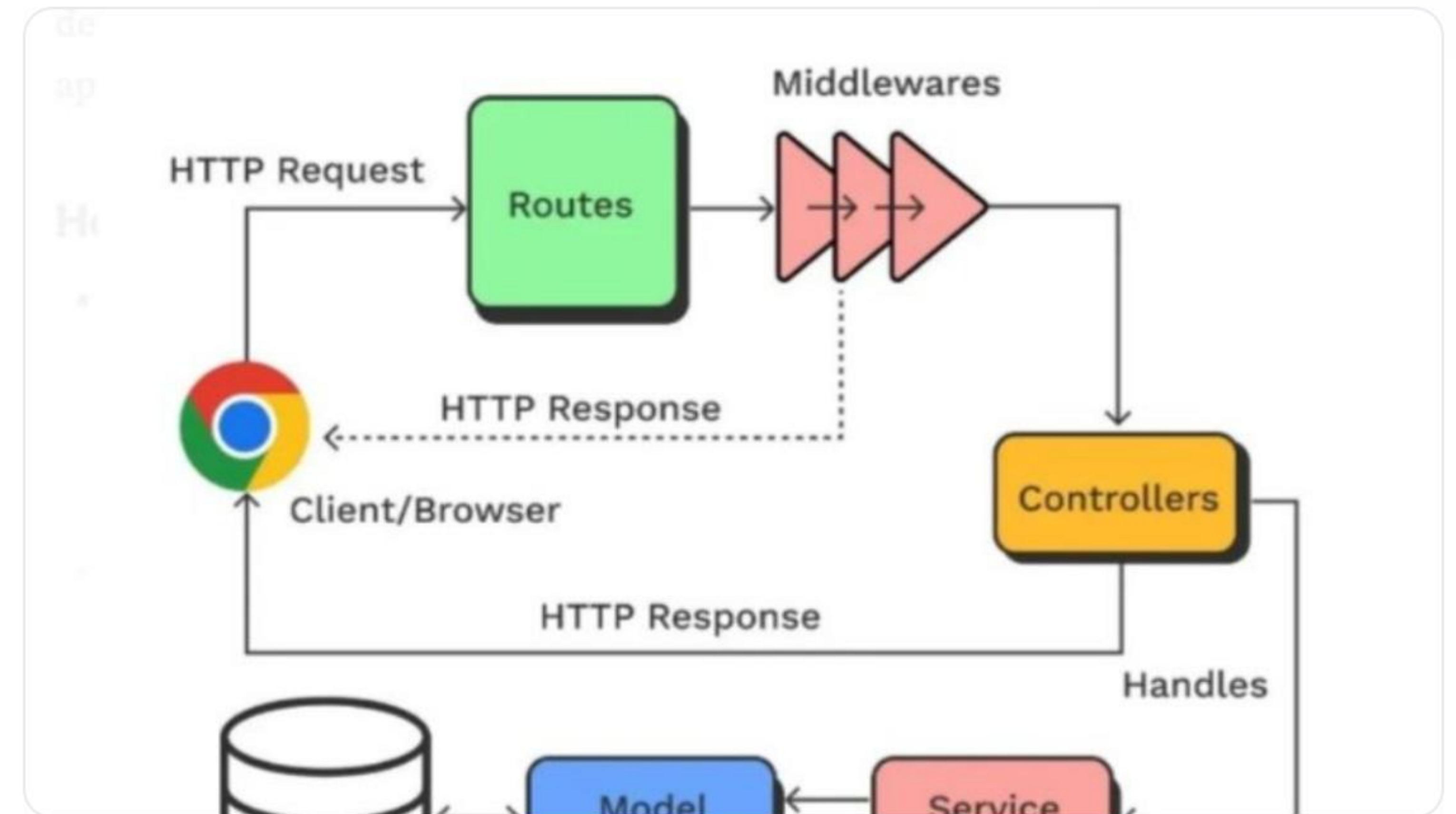


Binding

Dominar la transferencia de datos y el enlace de modelos en ASP.NET Core.

Repaso: Responsabilidades MVC

-  **Model:** El estado y las reglas de negocio.
-  **Controller:** El cerebro que orchestra el flujo.
-  **View:** La representación visual dinámica.



La Comunicación entre Capas

Para integrar las capas, no basta con tener carpetas separadas; debemos definir **contratos de comunicación** claros:

Hacia la Vista

Uso de ViewModels para empaquetar datos específicos que la interfaz necesita renderizar.

Desde la Vista

Uso de Model Binding para capturar parámetros de formularios y URLs directamente en objetos C#.

Transferencia de Datos

Enviando información del Controlador a la Vista.

ViewData, ViewBag y TempData

Característica	ViewData	ViewBag	TempData
Tipo	Diccionario (Llave-Valor)	Objeto Dinámico	Diccionario Persistente
Ciclo de Vida	Solicitud actual	Solicitud actual	Persiste tras redirección
Uso sugerido	Títulos, mensajes sutiles	Configuraciones simples	Alertas tras POST (Redirect)

El Patrón ViewModel

A diferencia del modelo de base de datos, el **ViewModel** está diseñado exclusivamente para la vista:

-  Seguridad: Solo expone lo necesario.
-  Composición: Une múltiples modelos en uno.
-  Formateo: Datos listos para mostrar.

```
public class HomeViewModel {  
    public List<Producto> Destacados { get; }  
    public string SaludoUsuario { get; }  
    public int Notificaciones { get; }  
}
```

Ventaja del Tipado Fuerte

100%

Detección en Compilación

Al usar modelos fuertemente tipados con la directiva `@model` en Razor, Visual Studio detecta errores de sintaxis antes de ejecutar la aplicación, aumentando la robustez del sistema.

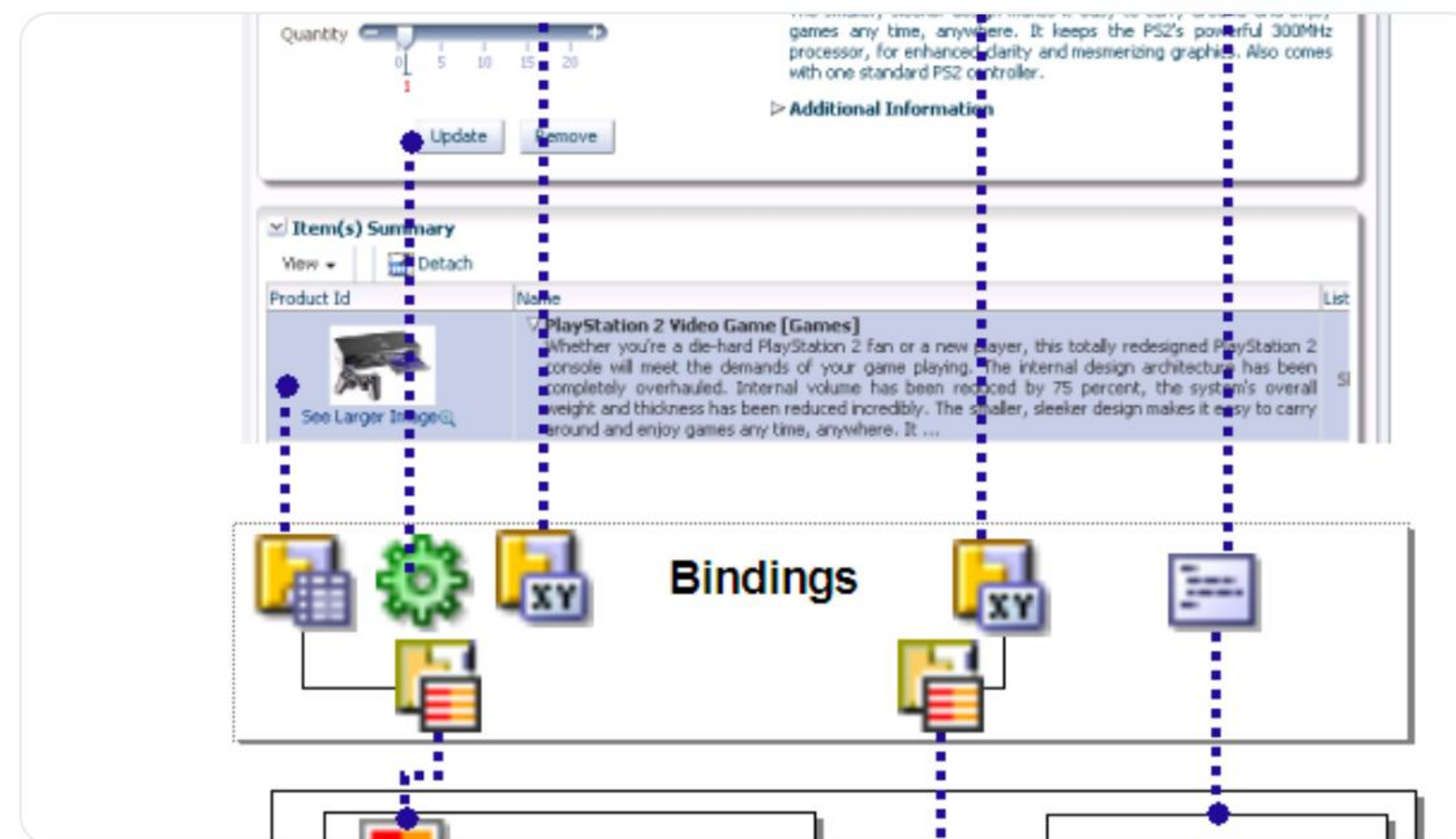
Model Binding

Capturando datos del cliente de forma automática.

¿Cómo funciona el Binding?

El motor de ASP.NET Core mapea los nombres de los inputs de HTML con las propiedades del modelo en C#:

- Lee la URL (QueryString).
- Lee el cuerpo (Form-Data / JSON).
- Instancia el objeto automáticamente.



Ejemplo de Binding en Código

```
// En el Controlador
[HttpPost]
public IActionResult Crear(Usuario nuevo) {
    if (ModelState.IsValid) {
        _context.Guardar(nuevo);
        return RedirectToAction("Index");
    }
    return View(nuevo);
}
```

Maapeo UI a Modelo

Los componentes visuales deben coincidir exactamente con los identificadores de backend:



name="Email"



```
public string Email
```



name="Fecha"



```
public DateTime Fecha
```



name="Activo"



```
public bool Activo
```

Showcase de Integración

Unificando las tres capas en un solo flujo.

1. Definición de la Entidad

Nuestra clase de dominio que define la estructura del dato:

```
public class Estudiante {  
    [Required, StringLength(10)]  
    public string Carne { get; set; }  
  
    [Required]  
    public string Nombre { get; set; }  
}
```

2. Controlador (Acción GET)

Preparamos la vista enviando un modelo vacío o datos iniciales:

```
public IActionResult Registro() {  
    return View(); // Carga el formulario limpio  
}
```

3. Vista Razor Integrada

Vinculamos la vista con el tipo Estudiante:

```
@model Estudiante  
  
asp-action="Guardar">  
    asp-for="Carne" />  
    asp-for="Nombre" />  
Inscribir
```

4. Controlador (Acción POST)

Recibimos y validamos el objeto íntegro enviado por el usuario:

```
[HttpPost]
public IActionResult Guardar(Estudiente data) {
    if (!ModelState.IsValid) return View(data);

    _database.Add(data);
    return RedirectToAction("Exito");
}
```

Uso de la Capa de Servicio

Para evitar controladores "gordos", delegamos la lógica a servicios:

Service Layer

Contiene la lógica pesada, cálculos y acceso a APIs externas.

Repository

Especializado únicamente en persistir y recuperar datos.

Integración de Servicios en C#

```
public class RegistroController : Controller {  
    private readonly IEstudianteService _service;  
  
    public RegistroController(IEstudianteService service) {  
        _service = service; // Inyección de Dependencias  
    }  
}
```

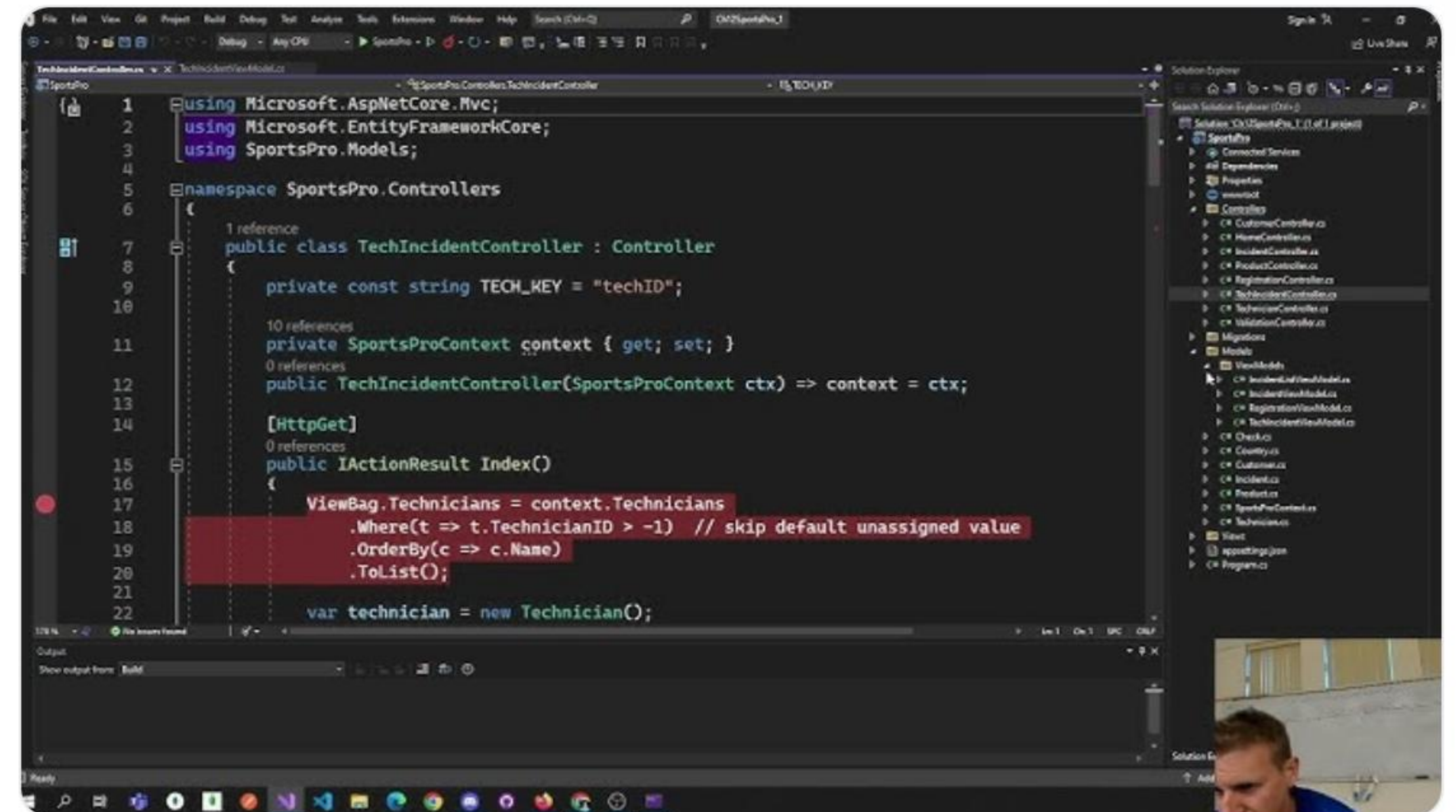
Mejores **Prácticas**

Escribiendo código profesional y mantenible.

El Patrón DTO (Data Transfer)

Jamás envíes tus entidades de base de datos directamente a la API o Vista:

- 🔒 Oculta IDs internos.
- 🛡️ Protege contra "Over-posting".
- 🚀 Mejora el rendimiento del tráfico de red.



The screenshot shows a Visual Studio IDE with a C# file named `TechIncidentController.cs`. The code is as follows:

```
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.EntityFrameworkCore;
3 using SportsPro.Models;
4
5 namespace SportsPro.Controllers
6 {
7     1 reference
8     public class TechIncidentController : Controller
9     {
10         private const string TECH_KEY = "techID";
11
12         10 references
13         private SportsProContext context { get; set; }
14
15         0 references
16         public TechIncidentController(SportsProContext ctx) => context = ctx;
17
18         [HttpGet]
19         0 references
20         public IActionResult Index()
21         {
22             ViewBag.Technicians = context.Technicians
23                 .Where(t => t.TechnicianID > -1) // skip default unassigned value
24                 .OrderBy(c => c.Name)
25                 .ToList();
26
27             var technician = new Technician();
28         }
29     }
30 }
```

The code demonstrates the use of AutoMapper (implied by the comment and the mapping logic) to map database entities to a DTO. The `Index` method filters out unassigned values and orders the results by name before returning them as a list. The solution explorer on the right shows the project structure, including `Controllers`, `Models`, and `Views`.

Limpieza de Vistas



No hagas consultas SQL en la vista.



Evita lógica compleja de IF/ELSE.



Usa modelos preparados
(ViewModels).

Valor: **Solidaridad**



Construyendo juntos un código excelente.

Colaboración en Capas

Estructurar el código en capas no es solo por orden, es por **solidaridad con tu equipo**:

- 🤝 Facilita el trabajo paralelo.
- 👥 Permite que especialistas (frontend/backend) colaboren sin bloqueos.
- ❤️ Reduce la frustración al depurar errores ajenos.



Pensamiento Social en Ingeniería

"El primer paso en la evolución de la ética es la solidaridad con los demás seres humanos."

— Albert Schweitzer

Integridad de la Arquitectura

100%

Código Modular

Una arquitectura solidaria permite que el software sea accesible para cambios futuros, asegurando la longevidad del proyecto y el bienestar de quienes lo mantienen.

Resumen de la Integración

Acción	Capa Responsable	Tecnología Clave
Captura de datos	Vista	HTML Form + asp-for
Mapeo a C#	Controlador	Model Binding
Validación lógica	Modelo / Servicio	DataAnnotations

Integración en Proyecto Final

En su proyecto final de laboratorio, la integración correcta de capas será evaluada estrictamente:

-  Estructura de carpetas por capas lógicas.
-  Uso de ViewModels en todas las pantallas.
-  Manejo correcto de errores mediante ModelState.

¿Preguntas?

Integración de capas y flujo de datos MVC.

Recursos Adicionales

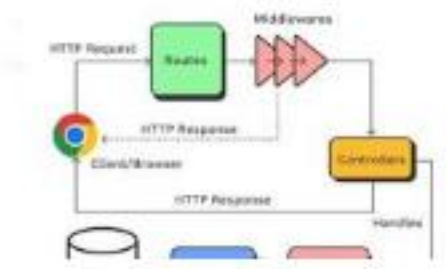
-  Documentación oficial de ASP.NET Core MVC.
-  Tutoriales de Inyección de Dependencias.
-  Horario de dudas en el foro UEDI.

¡Gracias por su atención!

Continuaremos con la implementación práctica de formularios complejos.

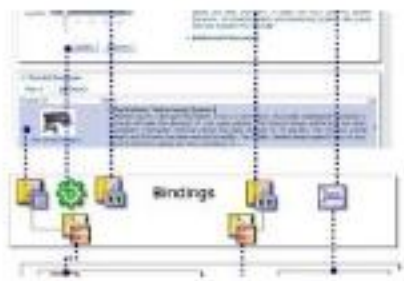
IPC 2 | Laboratorio | Ingeniería USAC

Image Sources



https://ik.imagekit.io/upgrad1/abroad-images/imageCompo/images/flow_of_data_in_MVC_architecture5C7CGV.jpg

Source: www.upgrad.com



https://docs.oracle.com/cd/E28271_01/web.11111/e28164/img/adf_bindings.png

Source: docs.oracle.com



https://i.ytimg.com/vi/MteRVvq3dOQ/hq720.jpg?sqp=-oaymwE7CK4FEIIDSFryq4qpAyOIARUAAAAAGAEIAADIQj0AgKJD8AEB-AH-CYAC0AWKAgwIABABGH8gMCg3MA8=&rs=AOOn4CLBggi4vrJVOTSp17quJcqf_ogHXNw

Source: www.youtube.com



https://meduzzen.com/wp-content/uploads/2026/04/1776505139730_Team-collaborating-in-open-plan-workspace-1.jpeg

Source: meduzzen.com